

Trabalho Prático I

DCC207 - Algoritmos II

Caio César Moraes Costa
2024005882
caiocesarmc@ufmg.br

Leonardo Barreto Gaião
2024005491
leonardobg@ufmg.br

May 3, 2026

Sumário

1	Introdução	1
2	Modelagem	1
3	Implementação	1
3.1	Conversão das localizações	1
3.2	Criação da árvore	1
3.3	Busca em área	1
3.3.1	Quadrilátero	2
3.3.2	Círculo	2
4	Interface	2
4.1	Layout	2
4.2	Mecanismos de Callback	3
4.3	Geolocalização	3
5	Resultados	3
6	Considerações finais	4
7	Referências	4

1 Introdução

O objetivo desse trabalho foi a implementação de um sistema para determinar os bares e butecos declarados na listagem Comida Di Buteco 2026, dentro de uma área determinada pelo usuário. O projeto como um todo deve ser feito em Python, e a interface em formato web, através da biblioteca `dash-leaflet`.

2 Modelagem

A natureza do problema leva a se considerar uma *Kd Tree* em 2 dimensões como uma estrutura de dados viável. Essa estrutura permite a busca de pontos pertencentes a uma região de forma eficiente, ao abandonar o teste de inclusão para todos os pontos, e em seu lugar, podar aqueles que certamente não estão dentro, enquanto que se obtêm todos aqueles que claramente estão inclusos.

As duas principais formas para a área, retangular e circular, são bem otimizadas para a busca na árvore, com a primeira representando uma busca em intervalo base, e a outra sendo uma adaptação dessa utilizando a distância euclidiana ao invés da comparação de coordenadas para determinar a inclusão.

Quanto a estrutura da *Kd Tree*, foi optado por incluir os pontos apenas nas folhas, ao invés de utilizá-los como os nós divisores do espaço. Essa escolha foi feita para que os nós intermediários representassem as regiões espaciais em si, o que facilitou a busca no caso da área circular.

3 Implementação

3.1 Conversão das localizações

Os bares que estão no dataset importado e a localização central da área que o usuário escolhe então em endereços reais, enquanto que a *Kd Tree* lida com pontos de duas dimensões. Uma conversão é feita para traduzir os endereços em pontos geográficos, onde o eixo X é dado pela latitude e o eixo Y pela longitude. Esse é um processo de custo $O(n)$, onde n é a quantidade total de pontos.

3.2 Criação da árvore

Como todos os pontos são conhecidos previamente, a construção da árvore é feita de uma vez, sem a preocupação de inserir nós posteriormente. A árvore então é montada de forma balanceada, ao dividir sucessivas regiões nos seus pontos medianos, alternando os eixos de referência. Os nós folhas armazenam os pontos existentes, e os nós intermediários têm a região (um retângulo) que contém todos as regiões e pontos de seus filhos. O nó raiz representa a divisão inicial da região no meio em relação ao eixo X.

Para cada nível que a árvore possui ($\log n$), é realizada uma ordenação com custo $O(n \log n)$ para obter a mediana e separar os pontos quanto a ela. Ainda que o número de ordenações por nível seja 2^k , onde k é o nível atual, cada ordenação nesse nível atua sobre $\frac{n}{2^k}$ items. Resolvendo a equação de recorrência pelo teorema mestre, temos que a complexidade total de construir a árvore é $O(n \log^2 n)$.

3.3 Busca em área

A mesma lógica utilizada em **3.1** é utilizada para converter o endereço que o usuário fornece em um ponto, que não necessariamente precisa existir previamente. Esse ponto se torna o centro de uma região definida por um tamanho: ou um quadrado com uma certa diagonal ou um círculo com um certo raio (todas as distâncias em quilômetros).

As buscas com a *Kd Tree* se aproveitam da sua estrutura para realizar duas otimizações massivas: a exclusão de ramos inteiros cujos pontos jamais estariam na área de interesse, e a inclusão de ramos cujos pontos garantidamente estão na área. Ambas eliminam a necessidade de comparação de todos os pontos com a zona de procura.

Ao final, uma lista posteriormente ordenada pela distância ao ponto central é retornada. Dependendo da área escolhida, a busca é efetuada de formas diferentes

3.3.1 Quadrilátero

Se usa o intervalo que o quadrilátero define: mínimo e máximo nos eixos X e Y. Partindo do nó raiz, checka-se:

- Se o nó for uma **folha**, inclua-o se suas coordenadas estão entre os mínimos e máximos da área de busca
- Se o nó define uma região **inteiramente contida** pela área de busca, inclua todos os filhos dele
- Se o nó define uma região **completamente fora** da área de busca, não explore esse ramo
- Se o nó define uma região mista, prossiga com a busca para os filhos desse nó

3.3.2 Círculo

Se passo o centro e o raio do círculo de busca. Como micro-otimização, as distâncias são calculadas ao quadrado, assim evitando o uso da raiz quadrada. Partindo do nó raiz, checka-se:

- Se o nó for uma **folha**, inclua-o se sua distância ao centro for menor que o raio
- Se o nó define uma região **totalmente dentro** do círculo de busca, inclua todos os filhos dele
- Se o nó define uma região que **não intersecta** a área de busca, não explore esse ramo
- Se o nó define uma região mista, prossiga com a busca para os filhos desse nó

Para ambos os casos, a complexidade da busca está relacionada a quantidade de regiões que são analisadas integralmente (as que não são podadas ou incluídas completamente), e da quantidade k de pontos que estão presentes na região. Ao final, a complexidade de tempo para a busca será $O(\sqrt{n} + k)$

4 Interface

4.1 Layout

A interface do sistema foi feita visando a simplicidade e é baseada em 3 componentes principais: o mapa, componente já existente no dashleaflet, o painel e uma tabela colapsável. A seguir, há uma explicação um pouco mais detalhada de como funcionam esses componentes:

- Mapa: É o container do Dash Leaflet responsável por carregar e permitir a interação com o OpenStreetMap, abaixo na hierarquia há o `dl.TileLayer()` que carrega o mapa efetivamente e o `dl.LayerGroup`, é a camada em que os markers são postos.
- Painel: É o componente que recebe a entrada do usuário. Ele é composto por dash core components, especificamente `dcc.RadioItems`, `dcc.Input` e `dcc.Button`, o primeiro é usado para escolher o "modo" de busca: quadrado ou círculo. Os inputs capturam o endereço que é usado como centro e o tamanho da diagonal ou raio, o botão serve para invocar a função `search`.
- Tabela: A mesma guarda os resultados da pesquisa ordenadas em ordem crescente em relação a distância. Diferente dos outros componentes, esse é feito sem utilizar componentes do dash, pois apesar do mesmo oferecer algo similar, a manipulação: torná-la colapsável e interativa foi mais fácil utilizando html diretamente. Como alternativa foi usado basicamente html e css para implementar a mesma, com destaque ao `html.Th`.

4.2 Mecanismos de Callback

A interatividade da aplicação é baseada em callbacks. Isso permite que o estado da interface seja atualizado de forma assíncrona sem que seja necessário recarregar a página.

- `update_label`: Este callback é responsável por mudar o título do segundo input. Ele monitora o valor do seletor de modo (Quadrado ou Círculo). Função: Altera o texto exibido para o usuário entre "Raio (km)" e "Diagonal (km)", adequando o nome ao modelo de busca escolhido.
- `search`: É o callback principal de visualização no mapa. Ele é acionado pelo botão de buscar e coordena o que aparece na tela. Função: Transforma o endereço em coordenadas reais e consulta a árvore de busca. Ele retorna para o mapa os marcadores dos bares encontrados, um marcador para o centro da busca e o desenho da área pesquisada (círculo ou retângulo).
- `toggle_tabela`: Gerencia o estado visual da tabela lateral que contém a listagem dos bares. Função: Utiliza a contagem de cliques para determinar se a tabela deve estar aberta ou fechada, alterando sua visibilidade e o sentido do ícone no botão de controle.
- `atualiza_tabela`: Trabalha junto com a busca, mas foca em organizar os dados de forma escrita na lista lateral. Função: Calcula a distância exata de cada bar em relação ao ponto de origem, ordena os resultados do mais próximo ao mais distante e gera as linhas da tabela com o nome, endereço e quilometragem de cada estabelecimento.

4.3 Geolocalização

Além do uso do Nominatim novamente, usado para converter o endereço fornecido pelo usuário em um latitude e longitude. Um detalhe importante da interface é que área de busca é delimitada no modo quadrado usando a função `distance` do `geopy`, isso acontece porque a terra tem um formato não euclidiano e não era possível obter facilmente o quão distorcida seria a área usando um cálculo simples, mesmo que com o limite de Belo Horizonte, provavelmente não seria uma grande distorção.

5 Resultados

Alguns exemplos do funcionamento do sistema:

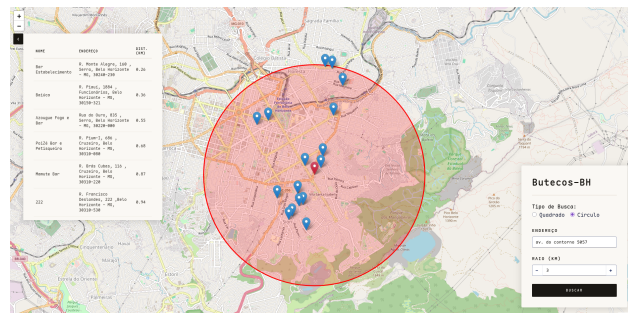


Figure 1: Busca circular

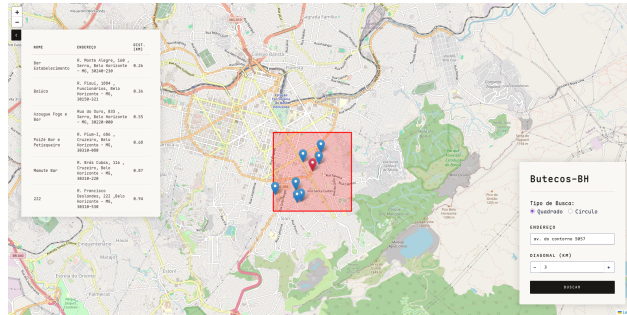


Figure 2: Busca retangular

6 Considerações finais

A realização desse TP foi bastante importante por além de focar na implementação de algoritmos discutidos em sala, ter sido um projeto que objetiva construir um produto "completo" no final, permitindo que praticássemos coisas como html, git, e uso de APIs para interagir com OpenStreetMap, sendo uma aplicação que pode vir a ser útil para alguém e que ajudou a desenvolver habilidades em vários aspectos do desenvolvimento de software.

7 Referências

- Advanced Algorithms and Data Structures, Marcello La Rocca
- (Slide) Range Searching And Kd-Trees, Utrecht University